Reviewer Pack — Minimal Rank of Hodge Structures over Tseitin Resolution Tre…

Entry 0539ef4db5de · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 06:43:03 UTC

Minimal Rank of Hodge Structures over Tseitin Resolution Trees

Entry ID: `0539ef4db5de` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 06:43:03 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Tseitin Resolution Trees

Statement:

> [‘For every Tseitin resolution tree C with n leaves, the minimal rank of its associated Hodge structure is equal to log_2(n).’, ‘This invariant ψ(C) = log_2(n) holds for all resolution trees computing a polynomial-time verifiable function.’, ‘ψ(C) does not exceed log_2(n) for any other Tseitin resolution tree C.’]

Rationale (proposer’s reasoning):

> [‘Hodge structures provide a rich algebraic-geometric invariant that could capture the complexity of computation in a way not captured by traditional circuit complexity measures.’, ‘The conjecture links Hodge theory with the structure of computational problems, which may reveal hidden relationships between seemingly disparate areas of mathematics and computer science.’, ‘If true, this would provide a new tool for proving lower bounds on the complexity of Tseitin resolution trees.’]

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `41770d0d3d7d4880`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a Tseitin resolution tree C with n leaves, the associated Hodge structure’s minimal rank is considered supported if it equals log_2(n) ± 0.5 for all 30 seeds and the mean difference between computed ranks and log_2(n) is ≤ 1.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Hodge structure" AND "Tseitin resolution tree" AND minimal rank"` - `"polynomial-time verifiable function" AND "Hodge structure" AND Tseitin` - `"ψ(C) = log_2(n)" AND "algebraic geometry" AND "complexity theory"`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_tree(n):
        if n == 1:
            return ['x', 'y']
        left = generate_tseitin_tree(n // 2)
        right = generate_tseitin_tree(n - n // 2)
        return [f'NOT {left[0]}', f'{right[0]} OR {left[1]}']

    def hodge_rank(tree):
        if isinstance(tree, str):
            return 1
        else:
            return max(hodge_rank(subtree) for subtree in tree)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        tree = generate_tseitin_tree(n)
        rank = hodge_rank(tree)
        expected_rank = math.log2(n)

        if abs(rank - expected_rank) > 0.5:
            return {
                "metric_name": "Hodge Rank",
```

```python
                "metric_value": rank,
                "instances_tested": len(n_values),
                "conjecture_holds": False,
                "counterexample": f"n={n}, computed rank={rank}, expected rank={expected_rank}"
            }
        results.append(rank)

    mean_diff = sum(abs(r - math.log2(n)) for n, r in zip(n_values, results)) / len(n_values)

    return {
        "metric_name": "Hodge Rank",
        "metric_value": sum(results) / len(results),
        "instances_tested": len(n_values),
        "conjecture_holds": mean_diff <= 1,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        if not result["conjecture_holds"]:
            break
        results.append(result["metric_value"])

    if len(results) == len(seeds):
        mean_val = sum(results) / len(results)
        std_dev = math.sqrt(sum((x - mean_val)**2 for x in results) / len(results))
        support_fraction = len([r for r in results if abs(r - math.log2(n)) <= 0.5]) / len(results

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_val} std={std_dev} support_fraction={support_frac
        else:
            print(f"RESULT: FALSIFIED counterexample=\"not enough support\" first_failing_seed={se
    else:
        print(f"RESULT: INCONCLUSIVE reason=counterexample_found_first seed={seeds[results.index(r
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'Hodge Rank', 'metric_value': 1, 'instances_tested': 6, 'conjecture_holds': Fa

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_78dba7d6.py", line 84, in <module>
    print(f"RESULT: INCONCLUSIVE reason=counterexample_found_first seed={seeds[results.index(next(r
math.log2(n)) > 0.5))]}")
                                                                                             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

StopIteration
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test produced a counterexample where for n=5, the computed Hodge rank was 1, which is outside the expected range of log_2(n) ± 0.5. | next: Investigate why the counterexample occurred and refine the conjecture or test conditions to ensure accuracy.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11441
2	preregistration	ollama_remote	glm4:latest	0	0	5753
3	novelty	ollama_remote	glm4:latest	0	0	4878
4	novelty	ollama_remote	glm4:latest	0	0	5406
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15135
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7185
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9333
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8289
9	judge	ollama_remote	glm4:latest	0	0	8631

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 76050 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0539ef4db5de.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0539ef4db5de.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/0539ef4db5de.tar.gz` (if generated)